

Real-Time Fraud Network Detection Engine

Project Insight Report | Dataset: Kaggle Credit Card Transactions Fraud Detection (Sparkov Simulation)

1. Project Overview

This project builds a fraud detection system on simulated credit card transaction data covering 1 Jan 2019 to 31 Dec 2020, spanning 1,000 customers and 800 merchants. Beyond standard tabular classification, the project incorporates a **network perspective**: customers and merchants are modeled as nodes in a graph connected by transaction edges, allowing extraction of relational features (customer merchant-diversity, merchant customer-diversity) alongside behavioral and temporal signals.

Training transactions	1,296,675
Test transactions	555,719
Fraud rate (train)	0.58% (7,506 cases)
Customers / Merchants	1,000 / 800
Final model	Random Forest (tuned threshold)
Final F1-score (fraud class)	0.65

2. Exploratory Insights

2.1 Time of day

Fraud rate is roughly **20x higher during late-night hours (22:00-03:00)**, ranging 1.4%-2.9%, compared to a flat 0.1%-0.15% during daytime and evening hours. This became the basis for the *is_night* feature, which ended up as the third most important predictor in the final model.

2.2 Transaction amount

Fraudulent transactions average **\$531** versus **\$68** for legitimate ones - nearly 8x higher. However, raw amount alone is a blunt signal since spending habits vary widely across customers, which motivated a personalized anomaly feature (see Section 3).

2.3 Merchant category

Online/card-not-present categories carry the highest risk: *shopping_net* (1.76%), *misc_net* (1.45%), and *grocery_pos* (1.41%), versus under 0.2% for everyday categories like *food_dining* and *health_fitness*.

2.4 Geographic distance (ruled out)

Distance between customer home location and merchant location was tested as a candidate network/spatial feature. Result: virtually identical distributions for fraud vs. legitimate transactions (~76 km mean for both). This is a known artifact of the Sparkov simulator, which places merchant coordinates near the customer regardless of fraud status - a useful negative result that prevented wasted effort in modeling.

3. Feature Engineering & a Critical Bug Fix

The most valuable part of this project was diagnosing and fixing a subtle **temporal data leakage / distribution-shift bug** in the network and behavioral features - a realistic engineering challenge that materially changed model performance.

3.1 The behavioral anomaly feature (amt_zscore)

Rather than using raw transaction amount, a z-score was computed measuring how many standard deviations a transaction's amount deviates from that specific customer's own historical average. Result: legitimate transactions average a z-score near 0, while fraudulent transactions average **3.2** (median 2.26) - meaning fraud typically appears as spending 2 to 6+ standard deviations above a customer's normal pattern. This became the single most important engineered feature.

3.2 The bug: static vs. time-aware aggregates

Initial network features (customer_txn_count, customer_merchant_diversity, merchant_customer_diversity, merchant_fraud_rate) were computed as static totals separately on the train and test files. Because the test period is shorter and represents a later continuation of the same customers' histories, this made every customer look artificially "new" at the start of the test set (e.g. customer_txn_count averaged 1,819 in train vs. 781 in test - not a real behavior change, just an artifact of how the aggregates were computed).

Tree-based models (Random Forest, XGBoost) split hard on these count thresholds and were badly misled by the artificial discontinuity, while Logistic Regression's smooth linear weighting was comparatively robust to it. This explains why the tree models initially *underperformed* a simple linear baseline - counter to normal expectations for fraud detection tasks.

3.3 The fix: causal, expanding-window features

All customer/merchant aggregates were rebuilt as **cumulative, time-ordered, causal** features computed on the combined chronological timeline (train + test concatenated, sorted by timestamp, then split back). Each feature at a given transaction now uses only information available strictly before that transaction - mirroring how a real-time production system would actually compute features at scoring time, with no leakage.

Metric	Before fix	After fix
Random Forest AUC-PR	0.043	0.693
Random Forest ROC-AUC	0.887	0.959
XGBoost AUC-PR	0.060	0.632

Table 1: Impact of fixing the temporal leakage bug on tree-based model performance.

4. Model Comparison (Final, Corrected Features)

Model	ROC-AUC	AUC-PR
Logistic Regression	0.936	0.209
Random Forest	0.959	0.693
XGBoost	0.979	0.632

Random Forest was selected as the final model on the basis of AUC-PR (the appropriate metric for a 0.58%-positive-rate problem), despite XGBoost's higher ROC-AUC. AUC-PR reflects ranking quality on the rare class specifically and is a more faithful measure of usefulness in an operational fraud-review setting.

4.1 Feature importance (final Random Forest)

Feature	Importance
amt	39.7%
amt_zscore	15.3%
is_night	12.9%
trans_hour	10.1%
merchant_fraud_rate	5.2%
customer_merchant_diversity	3.2%
customer_txn_count	3.1%
trans_count_last_1h	2.6%
age / city_pop / merchant_customer_diversity / trans_month / trans_day combined	~7.9%

Behavioral (amt, amt_zscore) and temporal (is_night, trans_hour) features dominate, but network features (customer_merchant_diversity, merchant_customer_diversity) and the real-time velocity feature (trans_count_last_1h) each contribute meaningfully once computed correctly - together accounting for roughly 9% of total importance.

4.2 Operating threshold

Threshold	Precision	Recall
0.1	39.6%	75.8%
0.2 (recommended)	62.5%	66.6%
0.3	75.9%	59.2%
0.5	89.2%	48.2%
0.7	97.3%	33.9%

At the recommended threshold of 0.2, the model catches two-thirds of fraud cases while keeping roughly 2 in 3 flagged transactions genuinely fraudulent - a workable balance for a human review queue. Final classification report at this threshold: **precision 0.63, recall 0.67, F1 0.65** on the fraud class.

5. Network Visualization

A focused subgraph was built around 30 sampled fraud transactions and their connected customers/merchants (2,030 transactions, 50 fraud edges after including overlapping connections). Customers and merchants were rendered as nodes, with fraud transactions highlighted in red against a backdrop of normal transaction edges.

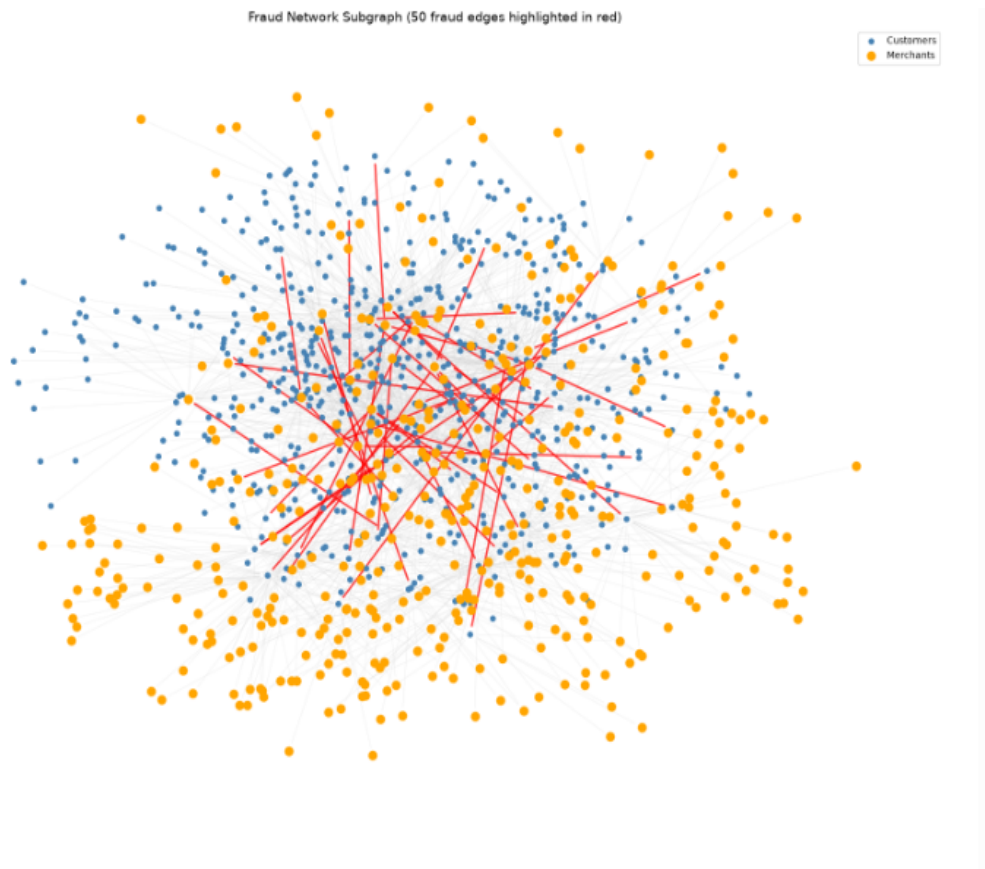


Figure 1: Fraud network subgraph. Blue = customers, orange = merchants, red edges = fraud transactions.

Fraud edges (red) visibly cluster in the center-left of the graph rather than scattering randomly across the network, suggesting certain customer-merchant connections are disproportionately implicated in fraud. This visual supports the project's core thesis: fraud is not purely an isolated per-transaction anomaly, but has a detectable relational/network signature worth incorporating into detection features.

6. Conclusion & Next Steps

Starting from raw transaction data, this project produced a Random Forest fraud classifier achieving 0.65 F1-score on a 0.58%-positive-rate problem, incorporating genuine network-derived features alongside behavioral and temporal signals. The single highest-value moment in the project was diagnosing a temporal leakage bug that had been silently capping tree-model AUC-PR at 0.04-0.06 - fixing it lifted Random Forest to 0.69 AUC-PR, a more than 10x improvement.

Recommended next steps:

- Build a real-time streaming simulation that replays test transactions in time order, updating rolling network/velocity features live and flagging fraud as it 'arrives'.
- Experiment with graph neural networks (e.g. GraphSAGE via PyTorch Geometric) using the customer-merchant bipartite graph directly as model input, rather than only hand-engineered graph statistics.
- Deploy the tuned Random Forest behind a lightweight API (e.g. FastAPI) with the threshold set to 0.2, logging flagged transactions for analyst review.
- Explore graph-community detection (e.g. Louvain) to automatically surface clusters of customers/merchants with elevated fraud concentration, rather than relying on manual sampling as done in Section 5.